

TimbreAudio: API quick reference

#include <TimbreAudio/TimbreAudio.h>, namespace `Timbre`, link the `TimbreAudio` project. Every call is thread-safe, and changes take effect together on the next `Update()`.

1. Engine

```
Timbre::AudioEngineConfig config; // every field is optional
config.SpatialMode = Timbre::SpatialMode::Binaural; // Panning = speakers (default), Binaural = headphones
config.Log = [](Timbre::LogLevel level, std::string_view text) { /* route to your logger */ };
Timbre::AudioEngine audio(config);

// once per frame
audio.SetListener({ .Position = camPos, .Forward = camForward, .Up = camUp, .Velocity = camVelocity });
audio.Update(); // publishes this frame's changes, runs OnEnd callbacks
```

Other config fields: `SampleRate` (0 = the device's), `MaxVoices` 512, `MaxAudibleVoices` 64 (the rest play silently as virtual voices), `MaxBuses`, `StreamBufferSeconds`, `BufferSizeMs`, `OutputDevice`, `Handedness` (`LeftHanded`), `SpeedOfSound`, `Interpolation` (`Cubic/Linear`), `RearHighCutDb` (6, dulls sounds behind you; 0 = off), `MasterLimiter`, `LoaderThreads` (0 = auto), `FileSystem` (your own `AudioFileSystem` for pak files), and `NoOutput` plus `audio.Render(buffer, frames)` for tests.

2. Sounds

```
Timbre::SoundDesc desc;
desc.Mode = Timbre::LoadMode::Stream; // Decompress: short SFX in memory | Compressed: keep encoded | Stream: from disk (music)
desc.Async = true; // load on loader threads; plays started early begin once it is loaded
desc.Bus = music; desc.Volume = 0.8f; desc.Looping = true;
desc.VolumeVariation = 0.2f; desc.PitchVariation = 1.0f; // random per play (volume factor, +- semitones)
desc.Priority = 200; desc.MaxInstances = 4; desc.LimitMode = Timbre::InstanceLimitMode::StealQuietest;
desc.LoopStart = 12.0f; desc.LoopEnd = 96.0f; // seconds; by default the file's LOOPSTART/LOOPEND tags
Timbre::Sound song = audio.LoadSound("res/Audio/Music.ogg", desc); // .ogg .wav .flac .mp3
Timbre::Sound fromMemory = audio.LoadSound(std::move(fileBytes), desc);
Timbre::Sound tone = audio.CreateSound(samples, 1, 48000); // procedural, floats in [-1, 1]
song.IsLoaded(); song.GetDuration(); song.GetInstanceCount(); song.StopAll(1.0f);
```

3. Voices (playing instances)

```
Timbre::PlayParams params;
params.Position = Timbre::Vec3(x, y, z); // makes it 3D (glm::vec3 converts implicitly)
params.Velocity = v; params.FadeIn = 0.5f; params.Delay = 0.1f; params.StartTime = 30.0f;
params.Volume = 1.0f; params.Pitch = 1.0f; params.Pan = -0.5f; // Pan applies to 2D voices only
params.Looping = true; params.Paused = false; params.Bus = sfx; params.OnEnd = [] { /* finished or stopped */ };
Timbre::Voice voice = song.Play(params); // or audio.Play(song, params)

voice.SetVolume(0.5f, 1.0f); voice.SetPitch(1.2f); voice.SetPan(0.3f); voice.SetLooping(false);
voice.SetPaused(true, 0.3f); voice.Stop(2.0f); voice.Seek(30.0f); voice.GetTime(); voice.IsPlaying();
voice.SetPosition(p); voice.SetVelocity(v); voice.SetDirection(forward);
voice.SetLowPass(800.0f); voice.SetHighPass(200.0f); // 0 = off; the low pass is the occlusion "muffle"
```

Handles are cheap to copy. They go inert when the voice ends, so keeping them around is safe.

4. 3D sound (`SoundDesc::Spatial`)

```
desc.Spatial.Model = Timbre::AttenuationModel::Inverse; // None | Inverse | Linear | Exponential | Custom
desc.Spatial.MinDistance = 1.0f; desc.Spatial.MaxDistance = 100.0f; desc.Spatial.Rolloff = 1.0f;
desc.Spatial.Curve = { { 0, 1.0f }, { 10, 0.5f }, { 40, 0.0f } }; // Custom: distance -> volume, linear in between
desc.Spatial.AirAbsorption = 1.0f; // highs roll off with distance (about 0.1 dB/m at 8 kHz), 0 = off
desc.Spatial.DopplerFactor = 1.0f; // needs voice and listener velocities
desc.Spatial.ConeInnerAngle = 90; desc.Spatial.ConeOuterAngle = 180; desc.Spatial.ConeOuterVolume = 0.3f; // with SetDirection
audio.SetSpatialMode(Timbre::SpatialMode::Binaural); // switch at runtime, e.g. from an options menu
```

Panning keeps loudness constant in every direction. Binaural mode models a spherical head (arrival-time and level differences between the ears, head shadow) for headphones. It has no elevation cues.

5. Buses

```
Timbre::Bus music = audio.CreateBus("Music");
Timbre::Bus world = audio.CreateBus("World");
Timbre::Bus reverb = audio.CreateBus("Reverb", world); //
child bus
music.SetVolume(0.5f, 1.0f); music.SetMuted(true);
world.SetPaused(true); // pause menu: holds its
voices
world.SetPitch(0.5f); // slow motion
music.SetLowPass(600.0f, 0.5f); // 0 = off
music.StopAll(1.0f);
audio.GetMasterBus(); audio.FindBus("Music");
```

6. Ducking

```
music.SetDucking({ .Trigger = dialog,
    .VolumeDb = -12.0f, .ThresholdDb = -40.0f,
    .AttackSeconds = 0.05f, .ReleaseSeconds = 0.6f });
music.ClearDucking();
```

The bus drops by `VolumeDb` while the trigger bus is louder than the threshold, then recovers.

7. Sends (reverb returns)

```
auto room = std::make_shared<Timbre::Reverb>();
room->SetDry(0.0f); // return bus: wet only
reverb.AddEffect(room);
desc.Sends = { { reverb, 0.3f } }; // default per play
voice.SetSend(reverb, 0.6f, 0.5f); // per voice, max 4
voice.GetSend(reverb);
```

A send taps the voice after its effects, and bypasses the voice's own bus volume. Make the return bus a child of the bus that should also control it.

8. Snapshots (mix states)

```
Timbre::SnapshotDesc paused{ {
    { music, -6.0f, 800.0f }, // bus, dB, low pass Hz
    { world, -12.0f, 500.0f } } };
Timbre::Snapshot s = audio.StartSnapshot(paused, 0.5f);
s.SetIntensity(0.5f, 0.2f); s.Stop(0.5f); s.IsActive();
```

Several snapshots can be active at once: their volumes add in dB and the lowest low pass wins. Bus volumes you set yourself are never overwritten.

9. Variations

```
Timbre::SoundContainer steps({ step1, step2, step3 },
    Timbre::ContainerMode::Random); // or Shuffle,
Sequence
steps.Play(params); // or steps.Next()
```

Random never plays the same sound twice in a row. Shuffle plays each sound once before reshuffling.

10. Effects

`BiquadFilter` (LowPass, HighPass, BandPass, Notch, Peak, LowShelf, HighShelf; shorthands `LowPassFilter`, `HighPassFilter`, `BandPassFilter`), `Delay`, `Reverb` (RoomSize, Damping, Wet, Dry, Width, PreDelay, DecayTime), `Compressor`, `Limiter`, `Distortion`. Attach with `bus.AddEffect(e)` or `voice.AddEffect(e)`, one owner per effect. Setters are atomic, so they are safe to call anytime.

Custom effect: derive from `Timbre::Effect` and override `Process(float* stereo, uint32_t frames)` (real-time safe: no locks, no allocation) and optionally `Initialize(sampleRate)`.

11. Misc

- `audio.GetStats()`: voices, audible and virtual counts, streams, CPU load, underruns, sound memory.
- `GetOutputDevices()`, `SetOutputDevice(name)`. Empty means the system default, and follows it when it changes.
- `Timbre::DecibelsToLinear`, `LinearToDecibels`, `SemitonesToPitch`.
- Sounds, voices and buses must not outlive the engine.